

CAREERHUBX · LOCAL DEVELOPMENT

How to Run This Two Terminals + Docker

A step-by-step runbook for starting the full CareerHubX stack on your machine — Postgres in Docker, the ASP.NET API, and the Next.js frontend.

 Docker · Postgres :5544

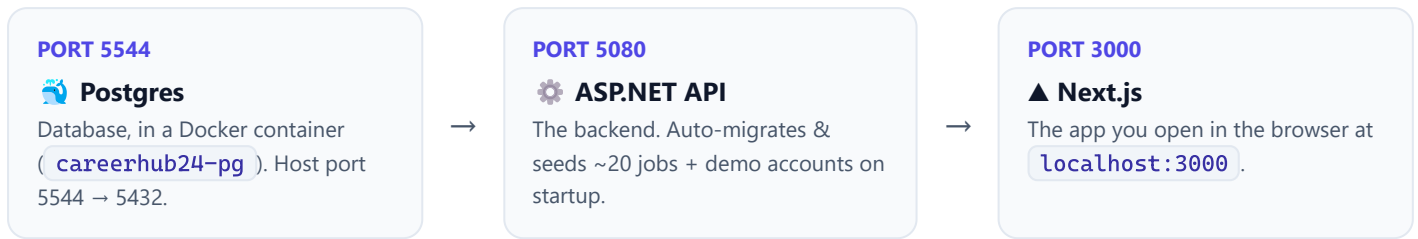
 .NET API :5080

 Next.js :3000

Windows · PowerShell · updated 2 July 2026

The big picture

CareerHubX is three programs that talk to each other. The frontend has **no mock data** — every read and write goes to the real backend, which in turn needs its Postgres database. So all three must be running.



Two terminals? Terminal 1 runs the backend (Docker Postgres + the .NET API). Terminal 2 runs the frontend. Docker Desktop runs in the background as a prerequisite — you start it once from the Start menu, not from a terminal.

Before you start — prerequisites

- **Docker Desktop** — installed and *running* (whale icon in the tray is steady, not spinning).
- **.NET SDK** — `dotnet --version` should print a version.
- **Node.js** — `node --version` (this project uses the version in `package.json`).
- **The backend repo** checked out next to this one at `C:\Users\Xolisa\assignment-2-4-backend` .



Terminal 1

Backend — Postgres + API

Step 1 — make sure Docker Desktop is up

Launch **Docker Desktop** from the Start menu and wait until it reports *Running*. Confirm the engine answers:

```
PS> docker version --format "{{.Server.Version}}"
```

If that errors or hangs: Docker sometimes boots unhealthy (every call returns HTTP 500). Fully quit Docker Desktop and relaunch `"C:\Program Files\Docker\Docker\Docker Desktop.exe"` , then re-run the command above until it prints a version.

Step 2 — start Postgres (in Docker)

```
PS> cd C:\Users\Xolisa\assignment-2-4-backend
PS> docker compose up -d
```

This starts the `careerhub24-pg` container on host port **5544**.

Already created it before? If the container exists but is stopped, just start it — don't recreate it (the name would clash):
`docker start careerhub24-pg`

Step 3 — run the API

In the *same* Terminal 1 (Postgres runs detached in the background):

```
PS> dotnet run --urls "http://localhost:5080"
```

On startup it auto-applies migrations and seeds roughly 20 South African jobs plus the demo accounts. **Leave this terminal running.**

Step 4 — verify the backend is alive

In a scratch terminal (or your browser), you should get `200 OK`:

```
PS> curl http://localhost:5080/health/ready
PS> curl http://localhost:5080/api/v1/jobs # returns the seeded jobs
```



Terminal 2

Frontend — Next.js

Step 1 — install dependencies (first time only)

```
PS> cd C:\Users\Xolisa\assignment-3.2
PS> npm install
```

Step 2 — start the dev server

```
PS> npm run dev
```

It reads `NEXT_PUBLIC_API_BASE_URL=http://localhost:5080` from `.env.local` and points every request at the API from Terminal 1.

Step 3 — open the app

Browse to `http://localhost:3000/jobs`. The first hit to each route compiles on-demand (10–30 s once), then it's fast.

Demo sign-in (job seeker): `demo.applicant@careerhub.co.za` / `DemoPass123!` — sign in before applying.

Quick reference

What	Where	Command / URL
Docker Desktop	Start menu	launch & wait for “Running”
Postgres	Terminal 1 · backend repo	<code>docker compose up -d</code>
API	Terminal 1 · backend repo	<code>dotnet run --urls "http://localhost:5080"</code>
Frontend	Terminal 2 · this repo	<code>npm run dev</code>
Open the app	Browser	<code>http://localhost:3000/jobs</code>

Troubleshooting

“Pages hang for a minute, then error / endpoints look dead”

Almost always the **backend stack isn't up**. With no mock data, if the API is unreachable the frontend retries for ~30 s several times before failing — so clicks appear frozen for a minute or two. Confirm Terminal 1 is running and `curl http://localhost:5080/api/v1/jobs` returns 200.

“Sign in / buttons do nothing; everything is slow”

Two environmental causes, not code bugs:

- **Stale dev servers.** Old `next dev` processes from previous sessions compete for CPU and file-watching. Kill them all and run exactly one:
`Get-Process node | Stop-Process -Force`
- **Cold first-compile.** On this machine the very first compile of a route can take up to ~70 s (warm ~11 s). Wait for the `✓ Compiled` line on the first hit to a route before deciding a link is broken.

Smoothest demo: build once and serve the precompiled app so there's no per-click wait —

`npm run build` then `npm run start` .

Docker container name already exists

Use `docker start careerhub24-pg` instead of `docker compose up` — the container is already created, it just needs starting.

Harmless noise: Watchpack `EINVAL ... C:\pagefile.sys` messages in the Next.js terminal are benign Windows file-watching warnings — ignore them.